

ЛР11 (3 пары / 6 часов)

«Рефакторинг: выносим команды меню в отдельные методы (и/или классы). Program.cs становится понятным»

Проблема, которую увидят дети

Сейчас `Program.cs` очень большой:

- много `if (input == "...")`
- много повторяющегося кода (показ списка, чтение id и т.п.)

Если так оставить:

- сложно исправлять ошибки
- легко что-то сломать
- "не видно структуру проекта"

Что делаем в ЛР11

Мы "разгружаем" `Program.cs` :

1. Вынесем повторяющиеся действия в **методы**
 2. (доп. для сильных) Вынесем команды в **классы** `Commands/*`
-

0) Итог ЛР11 (что должно быть готово)

✓ `Program.cs` стал короче и понятнее

✓ Есть отдельный файл `ConsoleUi.cs` (или `MenuHandlers.cs`), где лежат методы:

- PrintMenu
- PrintTasks
- TryReadInt

- Handlers для команд
(Add/List/Edit/Status/Delete/Import/Export/Backup/Stats/Log)

✓ Проект работает как раньше (функции не сломались)

✓ Есть документ:

- `docs/LR11_Report.md`
-

1) Теория (важное, развернуто)

1.1. Что такое рефакторинг

Рефакторинг — это улучшение структуры кода **без изменения поведения**.

То есть:

- программа делает то же самое
- но код становится чище

1.2. Зачем выносить в методы

Метод — это “кусоч программы с названием”.

Плюсы:

- легче читать
- меньше копипаста
- проще искать ошибки (в одном месте)

1.3. Принцип “одна ответственность”

- `Program.cs` должен управлять “общим потоком”: показать меню → вызвать нужную команду
 - а команду (например Add) лучше держать отдельно
-

2) Практика. Пара 1: создаём ConsoleUi и переносим туда вспомогательные функции

Шаг 1. Создать папку UI в проекте App

В `TaskTracker.App` :

1. Right click по проекту `TaskTracker.App`
 2. Add → New Folder
 3. Название: `UI`
-

Шаг 2. Создать файл ConsoleUi.cs

1. Right click по папке `UI`
 2. Add → Class...
 3. Название: `ConsoleUi.cs`
 4. Add
-

Шаг 3. Вставить код ConsoleUi (вспомогательные методы)

Откройте `ConsoleUi.cs` и вставьте:

```
using TaskTracker.Core.Models;

namespace TaskTracker.App.UI;

public static class ConsoleUi
{
    public static void PrintHeader()
    {
        Console.WriteLine();
    }
}
```

```

        Console.WriteLine("TaskTracker");
        Console.WriteLine("-----");
    }

    public static void PrintMenu()
    {
        Console.WriteLine("1) Добавить задачу");
        Console.WriteLine("2) Показать список задач");
        Console.WriteLine("3) Изменить статус задачи");
        Console.WriteLine("4) Удалить задачу");
        Console.WriteLine("5) Редактировать задачу");
        Console.WriteLine("6) Поиск по названию");
        Console.WriteLine("7) Фильтр по статусу");
        Console.WriteLine("8) Сортировка списка");
        Console.WriteLine("9) Сделать резервную копию (backu
p)");
        Console.WriteLine("10) Экспорт в файл (export)");
        Console.WriteLine("11) Импорт из файла (import)");
        Console.WriteLine("12) Статистика (отчёт)");
        Console.WriteLine("13) Экспорт отчёта в файл");
        Console.WriteLine("14) Показать последние строки лог
a");
        Console.WriteLine("0) Выход");
        Console.WriteLine("-----");
    }

    public static string ReadString(string prompt)
    {
        Console.Write(prompt);
        return Console.ReadLine() ?? "";
    }

    public static bool TryReadInt(string prompt, out int valu
e)
    {
        Console.Write(prompt);

```

```

        var text = Console.ReadLine();
        return int.TryParse(text, out value);
    }

    public static void PrintTasks(List<TaskItem> tasks)
    {
        if (tasks.Count == 0)
        {
            Console.WriteLine("Ничего не найдено.");
            return;
        }

        Console.WriteLine("Список задач:");
        foreach (var t in tasks)
        {
            Console.WriteLine($"{t.Id}. {t.Title} [{t.Statu
s}]]");
            if (!string.IsNullOrEmpty(t.Description))
                Console.WriteLine($"    Описание: {t.Descripti
on}");
        }
    }
}

```

Шаг 4. Проверка сборки

Build → Build Solution → Build succeeded.

Шаг 5. Коммит №1

Сообщение:

- refactor: add ConsoleUi helper for menu, input and list printing

Commit All → Push.

3) Практика. Пара 2: упрощаем Program.cs — используем ConsoleUi

Шаг 6. Открыть Program.cs

TaskTracker.App → Program.cs

Шаг 7. Добавить using на ConsoleUi

Вверх добавьте:

```
using TaskTracker.App.UI;
```

Шаг 8. Удалить старые функции TryReadInt / PrintTasks из Program.cs

Если они у вас были прямо в Program.cs — удалите их, чтобы не было дублей.

Шаг 9. В цикле заменить вывод меню на ConsoleUi.PrintMenu()

Найдите место, где вы печатаете меню строками `Console.WriteLine(...)`.

Замените на:

```
ConsoleUi.PrintHeader();  
ConsoleUi.PrintMenu();
```

И ввод команды:

```
var input = ConsoleUi.ReadString("Выберите пункт меню: ").Trim();
```

Шаг 10. Заменить TryReadInt / PrintTasks на вызовы ConsoleUi

Везде, где было:

- `TryReadInt(...)` → заменить на `ConsoleUi.TryReadInt(...)`
- `PrintTasks(...)` → заменить на `ConsoleUi.PrintTasks(...)`
- `Console.ReadLine()` для строк → лучше заменить на `ConsoleUi.ReadString(...)`

Пример:

```
var query = ConsoleUi.ReadString("Введите текст для поиска:");
```

Шаг 11. Быстрая проверка (F5)

Проверьте 3 вещи:

1. Добавить задачу
2. Показать список
3. Выйти

Если работает — значит рефакторинг не сломал проект.

Шаг 12. Коммит №2

Сообщение:

- `refactor: use ConsoleUi in Program for cleaner main loop`

Commit All → Push.

4) Практика. Пара 3: выносим хотя бы 3 команды в отдельные методы (handlers) + отчёт

Сделаем ещё шаг: вынесем обработчики команд в отдельный класс.

Шаг 13. Создать файл MenuHandlers.cs

В `TaskTracker.App`:

1. Right click по папке `UI` (или создайте папку `Handlers`)
2. Add → Class...
3. Название: `MenuHandlers.cs`

Шаг 14. Вставить “скелет” MenuHandlers

Откройте `MenuHandlers.cs` и вставьте:

```
using TaskTracker.App.UI;
using TaskTracker.Core.Models;
using TaskTracker.Core.Services;
using TaskTracker.Storage.Services;

namespace TaskTracker.App.UI;

public static class MenuHandlers
{
    public static void AddTask(TaskService service, JsonTaskStorage storage, AppLogger logger)
    {
        var title = ConsoleUi.ReadString("Введите название задачи: ");

        try
        {
            var task = service.Add(title);
            storage.Save(service.GetAll());
            Console.WriteLine($"Задача добавлена: #{task.Id} {task.Title} [{task.Status}]");
        }
    }
}
```



```

        logger.Info($"ADD id={task.Id} title=\"{task.Title}\"");
    }
    catch (ArgumentException ex)
    {
        Console.WriteLine("Ошибка: " + ex.Message);
        logger.Error("ADD failed: " + ex.Message);
    }
}

public static void ListTasks(TaskService service)
{
    var tasks = service.GetAll();
    if (tasks.Count == 0)
    {
        Console.WriteLine("Список задач пуст.");
        return;
    }

    ConsoleUi.PrintTasks(tasks);
}

public static void DeleteTask(TaskService service, JsonTaskStorage storage, AppLogger logger)
{
    ListTasks(service);

    if (!ConsoleUi.TryReadInt("Введите Id задачи для удаления: ", out var id))
    {
        Console.WriteLine("Ошибка: Id должно быть числом.");
        return;
    }

    var answer = ConsoleUi.ReadString("Точно удалить? (y/n) ");
    if (answer == "y")
    {
        storage.Delete(id);
        logger.Info($"Task with id {id} deleted.");
    }
    else
    {
        logger.Info($"Task with id {id} not deleted.");
    }
}

```

```

n): ").Trim().ToLower();
    if (answer != "y")
    {
        Console.WriteLine("Удаление отменено.");
        return;
    }

    try
    {
        service.Delete(id);
        storage.Save(service.GetAll());
        Console.WriteLine($"Задача с Id={id} удалена.");

        logger.Info($"DELETE id={id}");
    }
    catch (ArgumentException ex)
    {
        Console.WriteLine("Ошибка: " + ex.Message);
        logger.Error("DELETE failed: " + ex.Message);
    }
}
}

```

Мы вынесли 3 команды: Add, List, Delete. Этого достаточно для ЛР11. Остальные можно оставить в Program.cs или вынести тоже (для "5").

Шаг 15. Подключить handlers в Program.cs

Откройте `Program.cs` и:

- найдите блоки `if (input == "1")`, `if (input == "2")`, `if (input == "4")`
- замените их на вызовы:

Для "1"

```
if (input == "1")
{
    MenuHandlers.AddTask(service, storage, logger);
    continue;
}
```

Для "2"

```
if (input == "2")
{
    MenuHandlers.ListTasks(service);
    continue;
}
```

Для "4"

```
if (input == "4")
{
    MenuHandlers.DeleteTask(service, storage, logger);
    continue;
}
```

Шаг 16. Проверка (F5)

Обязательный сценарий:

1. Add (1) → добавили
2. List (2) → увидели
3. Delete (4) → удалили
4. List (2) → пусто
5. Лог (14) → появились записи

Шаг 17. Отчёт `docs/LR11_Report.md`

Создайте файл `docs/LR11_Report.md`:

```
# Отчёт ЛР11 – рефакторинг

## Что сделано
- Создан ConsoleUi (печать меню, чтение ввода, вывод списка)
- Program.cs стал короче: использует ConsoleUi
- Создан MenuHandlers и вынесены команды:
  - AddTask
  - ListTasks
  - DeleteTask
- Поведение программы не изменилось, всё работает как раньше

## Как проверить
1) Запустить (F5)
2) Добавить задачу
3) Показать список
4) Удалить задачу
5) Проверить, что всё сохраняется в JSON и лог пишется
```

Шаг 18. Коммит №3

Сообщение:

- `refactor: extract menu handlers and add LR11 report`

Commit All → Push.

5) Что сдавать (чек-лист)

- ✓ ссылка на GitHub
- ✓ `docs/LR11_Report.md`
- ✓ скрин/видео: Program.cs стал короче + работа Add/List/Delete

✓ минимум 3 коммита

6) Оценивание

"3" — вынесли ConsoleUi и использовали его

"4" — вынесли минимум 3 команды в MenuHandlers

"5" — вынесли почти все команды + аккуратные файлы/папки, всё работает
